Stewer Gabriel

DOCUMENTAÇÃO DO SISTEMA

Arquitetura

Camadas, como a página abre, mapa, fotos, edição simultânea, segurança e decisões técnicas.

Site e painel de stewergabriel.com
Visagismo e consultoria de imagem · Palmas, Tocantins · Desenvolvido por Gustavo Dev

Versão 2.2.0
08 de novembro de 2026

STEWER GABRIEL

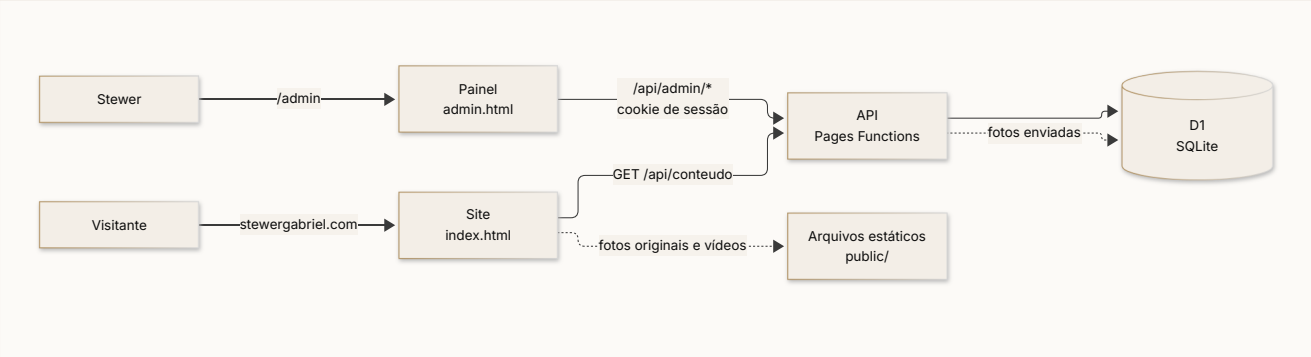
Arquitetura

Camadas, como a página abre, mapa, fotos, edição simultânea, segurança e decisões técnicas.

O site e o painel são um projeto só, publicado no Cloudflare Pages. O mesmo deploy leva três coisas:

- o **site** (`index.html` , React + Vite + Tailwind), que é o que o cliente vê;
- o **painel** (`admin.html` , React), uma segunda entrada do build, em `/admin` ;
- a **API** (`functions/` , TypeScript nas Pages Functions), que fala com o banco **D1** (SQLite gerenciado pela Cloudflare).

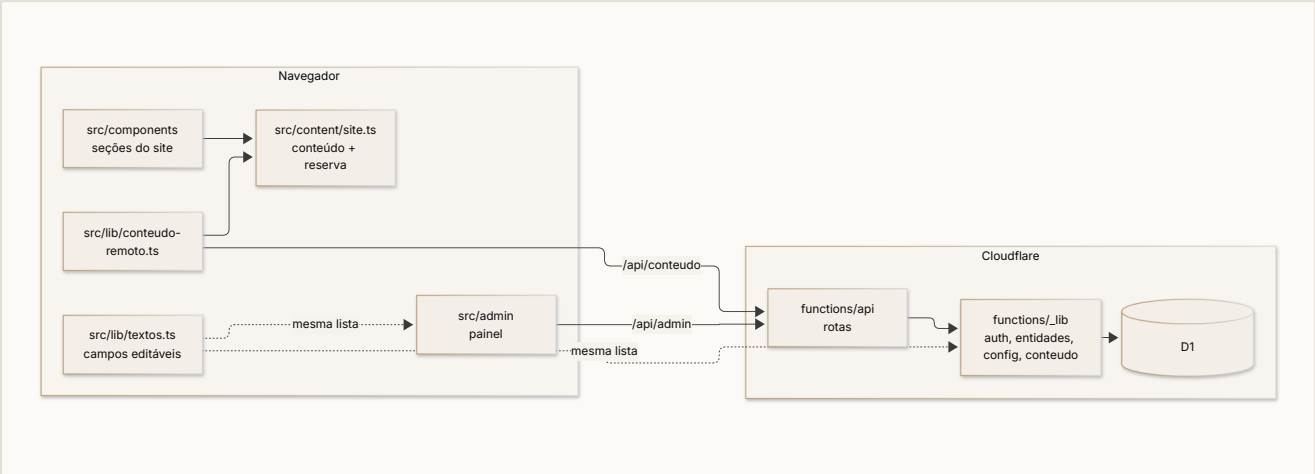
Tudo no plano gratuito. Não existe servidor ligado esperando: cada pedido à API acorda uma função na borda da Cloudflare, que responde em milissegundos.



Hospedagem e quem cuida de cada parte

PARTE	ONDE FICA	QUEM MEXE
Código	GitHub (`Gustavo16378/stewer-site`)	o mantenedor
Site, painel e API	Cloudflare Pages, projeto `stewer-gabriel`	deploy automático a cada push na `main`
Banco	Cloudflare D1 `stewer-site`	o painel (conteúdo) e as migrations (estrutura)
Segredo das sessões	variável `JWT_SECRET` do Pages, tipo Secret	o mantenedor, uma vez

Camadas



- `src/content/site.ts` guarda todo o conteúdo do site, com o texto original como reserva. Os componentes só leem daqui.
- `src/lib/conteudo-remoto.ts` busca `/api/conteudo` antes do primeiro render e troca o conteúdo de `site.ts` no lugar.
- `functions/_lib/entidades.ts` descreve as sete listas (tabela, campos, limites). Uma rota só (`/api/admin/[entidade]`) atende todas.
- `functions/_lib/config.ts` valida contatos e coordenadas.
- `functions/_lib/auth.ts` cuida de senha (PBKDF2) e sessão (JWT).

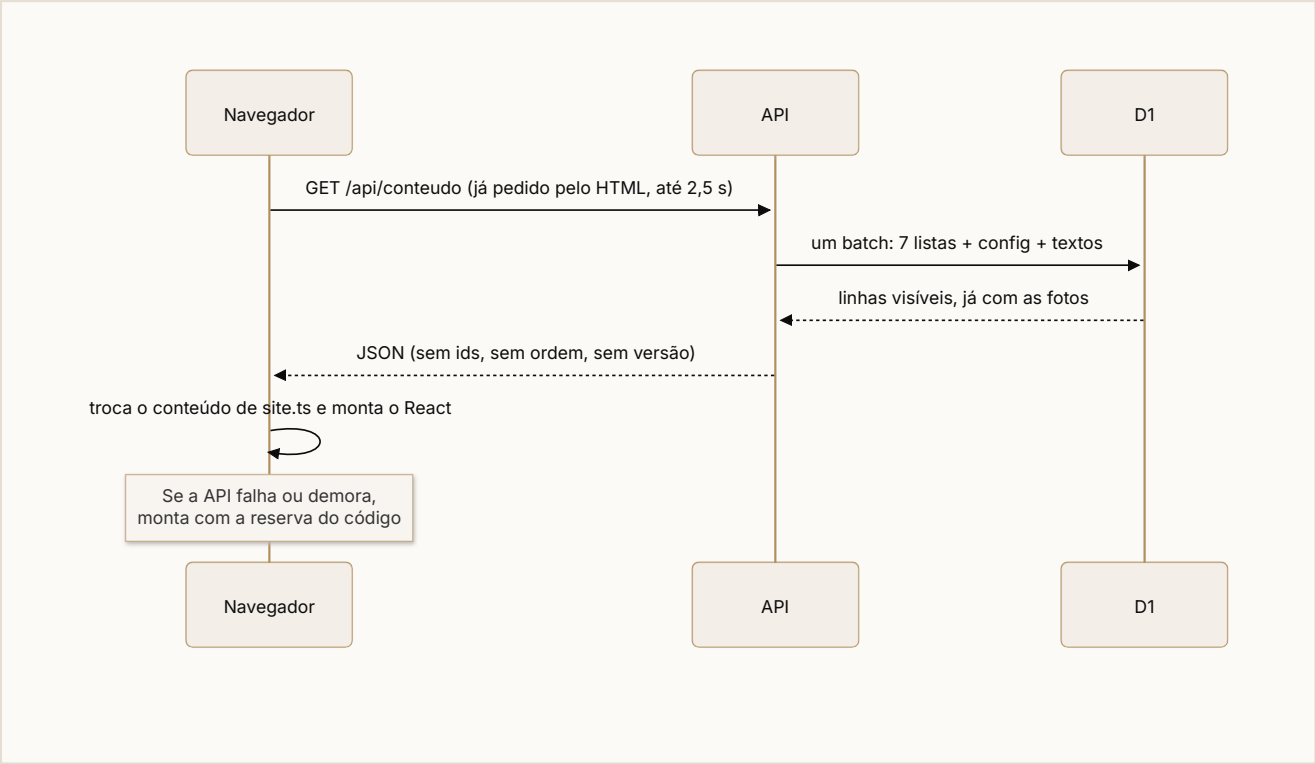
Uma fonte só para o que é compartilhado

`src/lib/textos.ts` é a lista de todos os textos fixos que podem ser trocados: chave, seção, rótulo, tipo e limite. Três lugares usam a mesma lista:

1. o **painel** monta a aba "Textos do site" a partir dela;
2. a **API** só aceita chaves que estão nela, com os limites dela;
3. o **site** aplica os valores salvos com a função `gravar` de cada campo.

Um campo novo entra num lugar e aparece nos três.

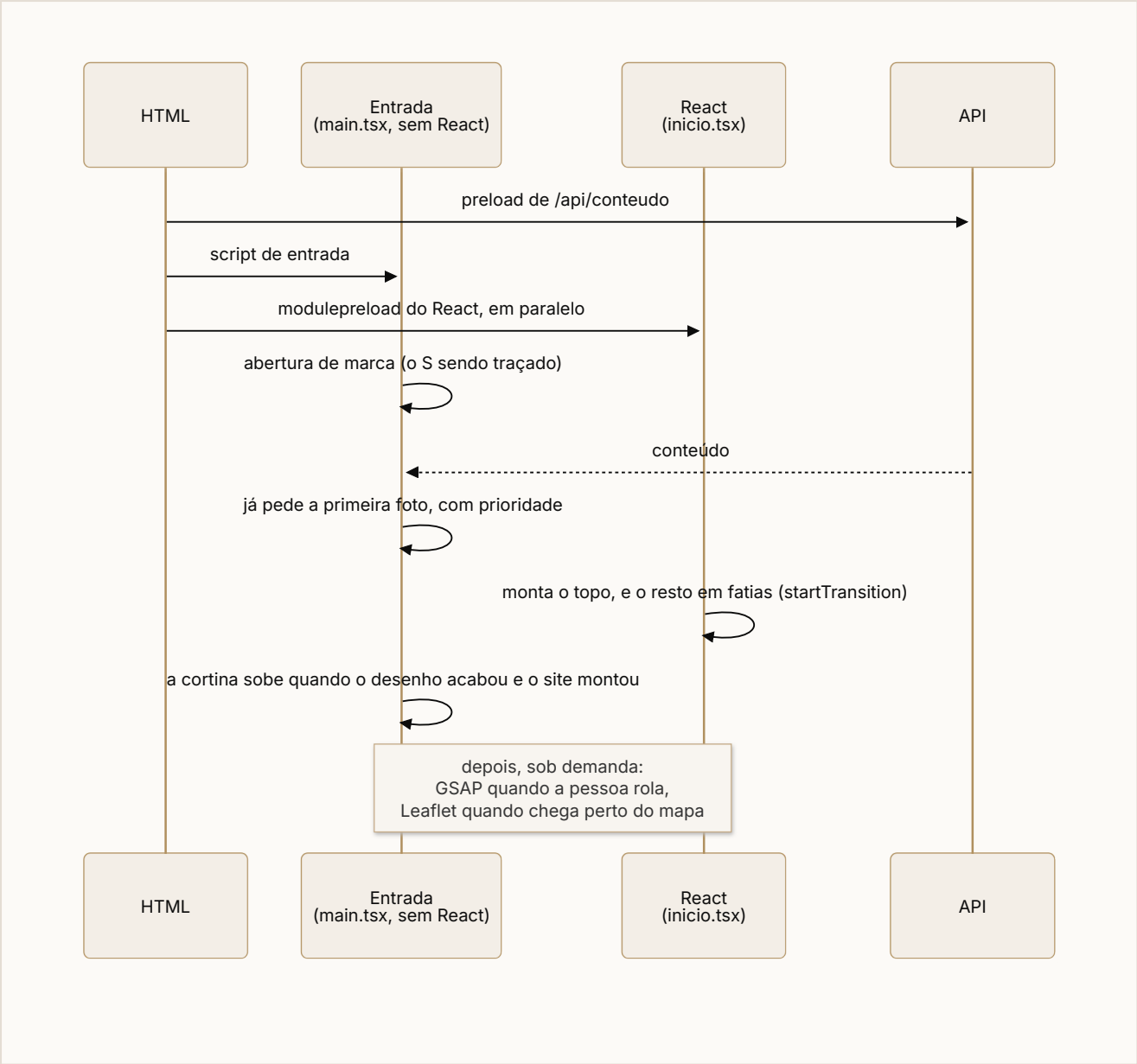
Fluxo de leitura do site



Trocar o conteúdo antes do primeiro render, e não depois com estado, é de propósito: o mapa e as animações de entrada leem o conteúdo uma vez, na montagem. O resultado é que o site com o banco fica idêntico ao de antes, pixel por pixel (conferido comparando capturas da página inteira no computador e no celular).

Como a página abre (desempenho)

A meta é a do Google para um celular com 4G lento: a maior coisa da tela (a foto da abertura) em até 2,5 s, nada pulando de lugar e o processador livre para responder. Medido a cada push (ver testes).



- **A entrada não tem React** (`src/main.tsx`): roda assim que chega, mostra a abertura de marca, pega o conteúdo e começa a baixar a primeira foto, enquanto o React chega em paralelo (`src/inicio.tsx` , pré-carregado por um plugin no `vite.config.ts`).
- **Abertura de marca** (`src/preloader.ts` e `.css`): o S do logo (vetor em `src/lib/logo.ts`) traçado com `stroke-dashoffset` , em CSS puro. Cobre a espera pelo conteúdo em vez de somar a ela. A cada carregamento; nunca para quem pede menos movimento; nunca prende o site (sobe em até 6 s).
- **O resto da página** monta numa transição do React, em fatias: montar tudo de uma vez travava o processador do celular por ~200 ms.
- **Movimento de rolagem** (`src/lib/movimento.ts`): GSAP com ScrollTrigger e SplitText, ligado por atributos no HTML (`data-revelar` , `data-contar` , `data-parallax`). Só é baixado quando a pessoa começa a rolar.
- **Mapa** (`src/lib/mapa-site.ts`): o Leaflet só é baixado a 600 px da seção.
- **Fontes** servidas pelo próprio site (`@fontsource`), só o subconjunto latino e os pesos usados.

Mapa

O fundo escuro do mapa tenta três fontes, nesta ordem (`src/lib/mapa.ts`):

1. **CARTO** `dark_all`, o visual original do design, quando há chave em `src/lib/mapa-fontes.ts` (desde setembro de 2026 a CARTO exige chave);
2. **OpenFreeMap** escuro, em vetor (MapLibre), sem chave nenhuma;
3. **OpenStreetMap** padrão escurecido por filtro, para quando o OpenFreeMap está fora ou o aparelho não tem WebGL.

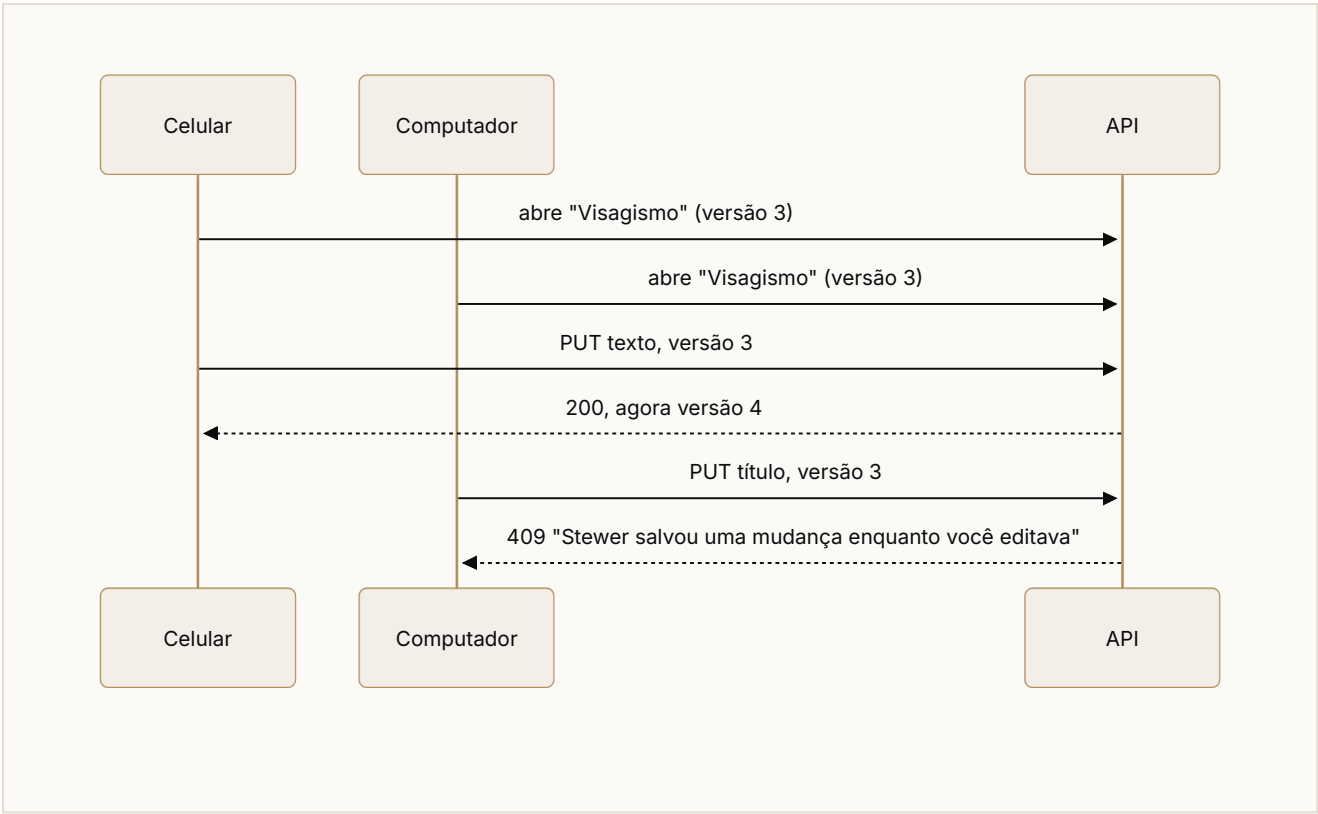
A troca é automática quando os pedaços do mapa falham. O mesmo fundo vale para o minimapa do painel, onde o Stewer cola o link do Google Maps ou arrasta o alfinete.

Fotos e vídeos

- **Fotos originais** continuam em `public/` e entram no banco só como endereço (`midias.caminho`). Não podem ser apagadas pelo painel.
- **Fotos novas** são reduzidas no navegador (até 1600px, WebP, até 1,4 MB) e guardadas dentro do D1 (`midias.dados`). A API entrega em `/api/midias/:id` com cache de um ano, porque o conteúdo de um id nunca muda: trocar a foto cria outro id.
- **Vídeos** ficam no código (`VIDEOS_PORTFOLIO` e `VIDEOS_AULAS`), porque não cabem numa linha do D1. O site intercala foto, vídeo, foto...

Edição simultânea

Cada item de lista tem uma coluna `versao`. O painel manda a versão que tinha quando abriu o item; a API grava com `WHERE versao = ?`. Se outra pessoa salvou no meio, a gravação não acha a linha e a API responde 409 dizendo quem salvou antes. Ninguém perde a mudança do outro sem saber.



O interruptor "No site" não manda versão (vale o último), porque é uma troca de um campo só e o resultado é sempre o que a pessoa vê.

Segurança

O resumo por risco do OWASP Top 10, com o teste de cada controle, está em `SECURITY.md`.

- Senha: PBKDF2-SHA256, 100 mil iterações, salt por usuário. O hash é calculado igual na API e no script de criar acesso.
- Sessão: JWT HS256 no cookie `sg_sessao`, `HttpOnly`, `SameSite=Strict`, `Path=/api`, `Secure` em HTTPS, 8 horas, apontando para uma linha da tabela `sessoes`. "Sair" apaga a linha; trocar a senha apaga as outras; senha temporária e acesso removido apagam todas.
- CSRF: além do `SameSite`, todo pedido que muda dados precisa vir da própria origem.
- Login trava por 15 minutos depois de 5 erros no mesmo e-mail.
- SQL: nomes de tabela e coluna só da lista fechada de `entidades.ts`; valores sempre por bind.
- Upload: tipo conferido pelos bytes, não pelo que o navegador diz. SVG é recusado (pode carregar script).
- Conteúdo: o React escapa todo texto; o único HTML montado à mão (o rótulo do marcador do mapa) passa por escape. Testado no E2E.
- Cabeçalhos: CSP sem script inline nem origem solta, `nosniff`, sem iframe de outro site, HSTS (`public/_headers` para as páginas, `functions/_lib/http.ts` para a API).
- SSRF: o link curto do mapa é aberto pelo servidor, só se for do Google.
- Dependências: `npm audit` na CI, Dependabot e CodeQL.

Decisões

DECISÃO	POR QUÊ
Pages Functions + D1, não um back em Java ou Node num servidor	servidor gratuito dorme e demora quase um minuto para acordar; aqui o custo é zero e a resposta é imediata
Mesmo repositório e mesmo deploy do site	um push publica tudo junto; sem CORS, sem segundo domínio
Fotos no D1, não no R2	o R2 pede cartão cadastrado na conta; foto reduzida cabe numa linha
Vídeos no código	não cabem no D1; o Stewer posta vídeo bem menos que foto
Conteúdo trocado antes do render	mantém o site idêntico ao original, sem piscar
Painel com a identidade do site	o Stewer reconhece a própria marca; o painel não parece um sistema genérico
Uma rota para as sete listas	as regras de cada campo moram num lugar só e são testadas uma vez
Sessão no banco, além do JWT	um JWT sozinho não tem como ser revogado: "Sair" precisa valer de verdade
Abertura de marca em CSS, não GSAP	aparece em ~300 ms, antes do React e do GSAP chegarem, e cobre a espera em vez de somar
GSAP só quando a pessoa rola	antes disso nada abaixo da dobra aparece; baixar e rodar o GSAP na abertura travava o celular ~250 ms
Fontes no próprio site	o CSS do Google Fonts bloqueava a primeira pintura e mandava o IP do visitante ao Google
Três fontes para o fundo do mapa	a CARTO passou a exigir chave e o site mostrava "API KEY REQUIRED"; agora nunca fica sem mapa